

EQUILIBRIUM FAIRNESS

A Runtime Governance Architecture for Monitoring and Correcting Fairness Drift in High-Impact AI Systems

Working Paper — Academic Edition

Version 1.1 · May 2026

Author

Maphi Bayolo

AI Governance Practitioner · ISO/IEC 42001 Foundation

Contact: Maphi.Bayolo@gmail.com

Keywords

AI fairness; runtime monitoring; algorithmic governance; drift detection; human oversight; EU AI Act; ISO/IEC 42001; negative feedback control; sociotechnical systems.

Abstract

This paper proposes **Equilibrium Fairness**, a runtime governance architecture for monitoring and correcting fairness drift in high-impact algorithmic decision systems. The architecture reframes fairness from a static property of a decision rule to a feedback property of a system over time, and specifies an operational loop, initialise, monitor, threshold, escalate. that connects regulatory principle to deployable practice. The contribution sits at the implementation layer beneath frameworks such as the EU AI Act, ISO/IEC 42001, and the NIST AI Risk Management Framework, providing connective tissue between high-level oversight obligations and the runtime instrumentation required to discharge them. The paper sets out the architecture's components, five universal conditions for equilibrium, a separation-of-roles principle in which automated monitoring is structurally distinguished from human-governed correction, a three-mode failure taxonomy, and a cross-domain mapping across seven sectors. A reference implementation in Python is described, with full code provided in a companion repository. The framework is presented as an operational architecture rather than a novel mathematical result; its value is interpretive and implementational. Limitations and avenues for empirical evaluation are discussed.

1. Introduction

1.1 The runtime governance gap

Contemporary regulatory and standards frameworks for artificial intelligence converge on a common set of obligations: high-impact systems must be assessed for bias, must be monitored throughout deployment, must support meaningful human oversight, and must be remediable when harms arise. The EU AI Act articulates these obligations at the level of regulatory principle; ISO/IEC 42001 and the NIST AI Risk Management Framework articulate them at the level of organisational management system. What neither layer specifies, and what neither layer can specify at its level of generality, is the *operational architecture* by which monitoring, threshold-setting, and correction are connected into a single governable runtime loop.

In practice the operational layer is fragmented. Bias audits are conducted periodically, often by external parties, against metrics that shift between audits. Threshold values for acceptable disparity are seldom defined in advance and seldom owned by a single accountable party. Correction mechanisms, where they exist, are typically retrospective and triggered by reputational or legal pressure rather than by structural detection. The result is a governance regime that is principled at the top and improvised at the operating layer, precisely the configuration that frameworks such as the EU AI Act's Article 14 (human oversight) and Article 15 (accuracy, robustness, cybersecurity) attempt to render insufficient.

1.2 Contribution

This paper proposes a runtime governance architecture for fairness drift, motivated by the gap above. The contribution is fourfold:

- First, a **single operational definition** of fairness as a feedback property: measurable, auditable, and domain general.
- Second, a **threshold-driven correction loop** that connects continuous monitoring to discrete corrective action with explicit accountability.
- Third, a **separation-of-roles principle**: monitoring and correction are held by structurally distinct functions, that aligns with EU AI Act human-oversight obligations and constitutional AI design patterns.
- Fourth, a **three-mode failure taxonomy** with mutually exclusive diagnostic categories, each mapping to a distinct intervention.

1.3 Scope and demarcation

The framework's contribution is operational and interpretive rather than analytic. Three boundaries are stated explicitly at the outset to forestall mischaracterisation. The formal expression $F(t) = E_0 - \sum \delta_i(t) + \sum \kappa_j(t)$, introduced in Section 4, is *an accounting identity rendered in systems-theoretic notation*; it is not a theorem and is not advanced as mathematically novel. The framework does not *solve* fairness, in the sense of selecting between competing fairness metrics; it provides the surrounding governance loop within which any such metric, demographic parity, equalised odds, calibration, individual fairness; may be operationalised. The framework does not replace domain-specific regulatory regimes; it offers a candidate implementation architecture beneath them.

The framework is currently at an early operational maturity stage and is presented as a working paper intended to support discussion, implementation, and empirical evaluation.

1.4 Structure

Section 2 situates the framework within the existing literature on algorithmic fairness, runtime monitoring, and human-oversight architectures. Section 3 develops the core thesis: fairness as a feedback property of a system over time. Section 4 specifies the operational architecture, including the loop's four stages, the equation that formalises them, and a visual architecture diagram. Section 5 sets out five universal conditions for equilibrium. Section 6 elaborates the runtime governance layer, including monitoring, escalation, oversight, and auditability. Section 7 maps the architecture across seven domains. Section 8 addresses regulatory alignment with the EU AI Act, ISO/IEC 42001, and the NIST AI RMF. Section 9 discusses limitations and avenues for empirical evaluation. Section 10 concludes. Appendix A presents a Python reference implementation; Appendix B applies the framework to recruitment as an illustrative implementation domain; Appendix C provides a glossary.

2. Related Work

2.1 Statistical fairness metrics

The technical fairness literature has converged on a small number of group-statistical criteria — demographic parity (Dwork et al. 2012), equalised odds and calibration (Chouldechova 2017; Kleinberg, Mullainathan & Raghavan 2017; Hardt, Price & Srebro 2016) — and has established the impossibility of satisfying multiple criteria simultaneously in non-degenerate cases (Chouldechova 2017). The present framework is agnostic to the choice of metric; its contribution is the monitoring-threshold-correction loop within which any such metric may be operationalised, rather than a new metric. The impossibility results inform the framework's design: because no single metric resolves fairness in all contexts, the architecture treats metric selection as a governance act (specifying δ) rather than a technical default.

2.2 Algorithmic auditing and runtime monitoring

Algorithmic auditing has matured from ad-hoc investigation (Buolamwini & Gebru 2018; Obermeyer et al. 2019) toward more structured approaches. Raji et al. (2020) propose internal auditing frameworks for ML systems; Mitchell et al. (2019) introduce model cards as standardised evaluation documentation; and Selbst et al. (2019) identify the risk of treating fairness as a purely technical property ('the portability trap'). Runtime monitoring of deployed ML systems has been addressed in the MLOps and model-observability literature, though typically focused on performance drift (accuracy, latency) rather than fairness drift specifically. The Equilibrium Fairness architecture extends this monitoring paradigm to fairness metrics, with the addition of a separation-of-roles principle and an explicit threshold-governance layer that the observability literature does not address.

2.3 Human oversight and governance architectures

The requirement for meaningful human oversight in high-risk AI systems is now codified in EU AI Act Article 14 and operationalised in ISO/IEC 42001 Clauses 8 and 9. The governance literature (Metcalf et al. 2021; Kroll et al. 2017) has identified a persistent gap between the principle of human oversight and its operational realisation: oversight obligations are stated at the level of organisational management but are rarely connected to runtime instrumentation. Veale & Binns (2017) note the difficulty of translating legal obligations into deployable technical controls. The present framework addresses precisely this gap: it proposes an operational architecture that renders human-oversight obligations actionable through a structured loop with named accountability at each stage.

2.4 Control-theoretic approaches

The use of control-theoretic metaphors in sociotechnical fairness analysis is not new: Friedler, Scheidegger & Venkatasubramanian (2021) and others have drawn on feedback-loop reasoning to model the dynamics of algorithmic decision-making. The present framework draws on negative-feedback control explicitly, but advances a governance claim rather than a modelling one: the contribution is not the observation that fairness has feedback dynamics, but the specification of a minimum-viable governance loop; initialise, monitor, threshold, correct, that connects that observation to deployable practice.

2.5 Positioning: what existing frameworks provide and do not provide

The table below locates the present contribution relative to the three principal normative frameworks now consolidating around high-impact AI. The architecture is designed to fill the operational gap between the regulatory obligation and the runtime instrumentation that discharges it.

Framework	What it provides	What it does not provide
EU AI Act	Legal obligations: risk management, human oversight, accuracy monitoring, transparency.	No runtime architecture connecting those obligations to deployed instrumentation.
ISO/IEC 42001	Management-system structure: planning, operation, evaluation, improvement.	No monitoring loop, no threshold governance, no separation-of-roles specification.
NIST AI RMF	Risk management functions: govern, map, measure, manage.	No operational implementation architecture beneath the four functions.
Equilibrium Fairness	Runtime governance loop: initialise, monitor, threshold, correct. Separation of roles. Failure taxonomy. Audit artefacts.	Not a compliance framework. Not a fairness metric. Requires empirical validation.

3. Core Thesis: Fairness as a Feedback Property

3.1 Reframing the object

Conventional fairness analysis in machine learning treats fairness as a property of a decision rule evaluated at a single point in time. A classifier is fair, or unfair, with respect to a chosen statistical criterion computed over a held-out dataset. This formulation has been productive. It has yielded the demographic parity, equalised odds, and calibration criteria that structure much of the technical fairness literature, but it is silent on what happens to a system once deployed.

In production, a fairness-evaluated system encounters distributional shift, feedback effects from its own decisions, operational drift in upstream data pipelines, and gradual realignment of human review processes. The point-in-time property established at evaluation is not preserved over operating time. A system that satisfies a chosen fairness criterion at deployment may violate the same criterion six months later without any change to its code, because the world the system operates in is not static and because the system itself reshapes that world through its decisions.

This paper accordingly reframes the object of fairness analysis from *the rule* to *the loop*: fairness is a feedback property of a deployed system, sustained or lost through the system's monitoring, threshold, and correction infrastructure over time.

Fairness is not a state — it is a feedback property.

3.2 Drift as the failure mode

If fairness is a feedback property, then the central failure mode is *uncorrected drift*: small distributional asymmetries that compound, over many decisions, into durable structural disadvantage. The reframing matters because it shifts the governance question from *how do we eliminate bias* to *how do we maintain equilibrium in a system that will always drift*. The first question invites unattainable static solutions; the second invites operational ones, monitoring, thresholds, correction, accountability.

This is the framework's formal statement of equilibrium:

Fairness is a dynamic equilibrium in which local differences among individuals may exist, but do not compound into durable advantages that enable domination, exclusion, or irreversible hierarchy.

4. Operational Architecture

4.1 Four stages

The architecture comprises four stages, each addressing one of the failure modes that the literature on deployed algorithmic systems has identified.

#	Stage	Symbol	Function
1	Initialise	E_0 — starting equity	Strip inherited advantage from inputs; equalise access conditions before evaluation begins.
2	Monitor	δ — drift signal	Detect distributional deviation from the equitable baseline using continuous, automated instrumentation.
3	Threshold	θ — escalation cut	Apply a pre-agreed threshold to the drift signal; breach triggers escalation.
4	Correct	κ — human correction	A governed, accountable, human-authorised corrective action proportionate to the breach.

4.2 Architecture diagram

The four stages are connected in a negative-feedback control loop. The diagram below summarises the architecture's operating flow. Stages 2 and 3 (monitoring and threshold-checking) may be automated; stage 4 (correction) must be human-governed.

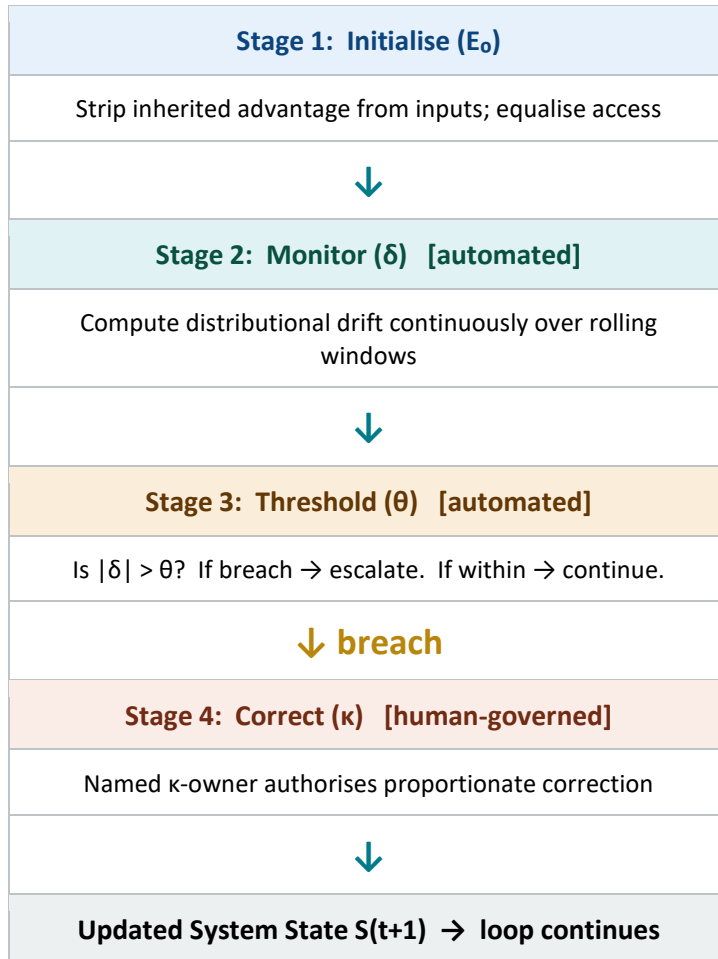


Figure 1. *The Equilibrium Fairness architecture. The loop runs continuously; the correction stage is the only stage that requires human governance. The architectural separation between automated monitoring (stages 2 and 3) and human-governed correction (stage 4) is the framework's load-bearing constraint.*

4.3 Formal expression

The state of the system at time t may be expressed as a balance of initial equity, accumulated uncorrected drift, and applied corrections:

$$F(t) = E_0 - \sum \delta_i(t) + \sum \kappa_j(t)$$

where

$F(t)$	the system's fairness state at time t
E_0	starting equity (initial conditions, equalised inputs)
$\sum \delta_i(t)$	accumulated uncorrected drift across i drift events to time t
$\sum \kappa_j(t)$	accumulated corrections across j correction events to time t

This expression is an **accounting identity**; a bookkeeping relation among quantities the architecture obliges its operator to name. It is not a theorem. Its purpose is disciplinary: it forces a system designer to specify E_0 , identify δ , set θ , and assign κ and its owner. The interpretive claim that many high-impact decision systems can be governed through a common monitoring-threshold-correction loop when their operating quantities are reducible to these four terms is the framework's substantive contribution and is the hypothesis that bears the weight.

The architecture's operational reading is direct: **a system is not failing because it is imperfect, it is failing when $F(t)$ is decreasing and no κ fires.**

4.4 Separation of roles: monitor $\neq$ decide

The architecture is anchored on a separation-of-roles principle. The monitoring function M , which computes δ and tests against θ , can be automated; in high-volume systems it must be, since periodic human audit is insufficient to detect drift between audit windows. The correction function κ must be human-governed: legitimacy in fairness correction cannot be delegated to an algorithm, and the accountability arc on which oversight regimes depend cannot be closed without an identifiable human decision-maker.

This principle aligns directly with the EU AI Act's Article 14 requirement that high-risk AI systems be designed to be effectively overseen by natural persons, with constitutional AI design patterns in which value alignment is approached through layered specification rather than monolithic optimisation, and with safety-engineering practice in which the sensing and actuating functions of a control system are placed under distinct accountability.

Monitoring may be automated. Correction must be governed. The architecture treats this distinction as load bearing rather than incidental.

5. Five Universal Conditions for Equilibrium

The architecture requires that all five of the following conditions be satisfied jointly. Satisfying four is not sufficient: each condition addresses a distinct failure mode, and the absence of anyone collapses the loop. The conditions are domain-general; their instantiation in any specific system is the four-question protocol set out in Section 7.3.

1	Initialise fairly	Strip inherited advantage from inputs. Remove any factor that correlates with identity rather than capability or need. The literature on input bias (Dwork et al. 2012; Selbst et al. 2019) gives this condition its empirical anchor; the architecture treats it as foundational because deviation from E_0 cannot be detected by downstream monitoring if E_0 was never set.
2	Define the range	Agree in advance what counts as acceptable spread — the threshold θ . Without a defined range, correction cannot be triggered, because there is no breach to detect. Threshold-setting is a governance act, not a technical one; it must be owned by an accountable party with the standing to bind the operating organisation.
3	Monitor continuously	Detect δ before it compounds. Periodic audits are insufficient on the simple ground that structural drift accumulates between audit windows, and the consolidation cost of accumulated drift grows non-linearly in the drift's age. Continuous monitoring is the architectural counterpart of the EU AI Act's Article 15 requirement for ongoing accuracy and robustness monitoring.
4	Correct early and proportionately	Late correction is both more costly and less stable than early correction. The institutional analogue is preventive medicine: keeping deviations within bounds is qualitatively preferable to repairing collapse after hierarchy has consolidated. Governance design should accordingly prioritise threshold detection rather than relying on crisis as the corrective trigger.
5	Separate roles	Monitoring and correction must be held by structurally distinct functions. Automated systems detect and flag; humans govern the correction. This preserves legitimacy, accountability, and auditability, and is the architectural foundation on which the framework's regulatory alignment is constructed.

5.1 Two structural asymmetries

The five conditions are flanked by two structural asymmetries that bear on governance design.

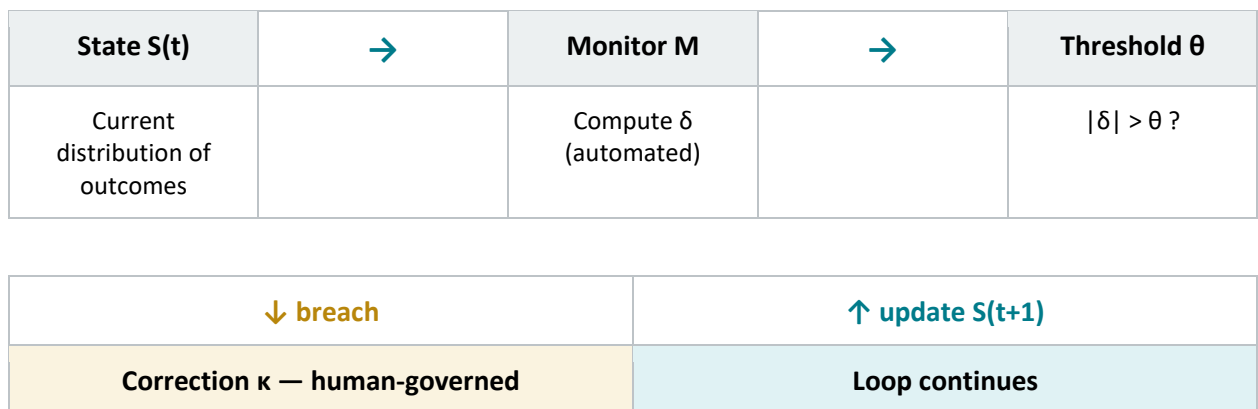
First, **late correction costs more**. Where monitoring and correction mechanisms are absent, systems can restore equilibrium only through substantial disruption: political crisis, institutional collapse, or coercive intervention. The asymmetry between proactive and reactive correction is qualitative, not merely quantitative; governance design should accordingly prioritise threshold detection rather than relying on crisis as the corrective trigger.

Second, **initialisation is irreversible**. Fairness cannot be retrofitted after disparities have already compounded. In practice this means that identity masking, structured assessment, and access equalisation must occur at the input stage. Once a structural hierarchy is entrenched, the correction cost is qualitatively higher and may exceed the political or institutional appetite to bear it.

6. Runtime Governance

6.1 The control loop

The architecture's runtime layer is a negative-feedback control loop. Negative feedback is the systems-theoretic mechanism by which equilibrium is maintained against perturbation: deviation from a reference range triggers a countervailing response that restores the system toward equilibrium. The operative loop is shown below.



6.2 Three failure modes

A system instrumented by this architecture can fail in exactly one of three ways. The taxonomy is forced-choice, the three modes are mutually exclusive within the equation, and the diagnostic value of the framework rests on the requirement that any observed fairness failure be assignable to one of these modes.

Failure mode	Mechanism	Intervention
E_0 never set	The system began with unequal inputs. Bias is structural from the first decision and cannot be detected as drift, because there is no equitable baseline to drift from. The system's fairness signal will be silent even as harm accumulates.	Redesign the intake process. Introduce structural equalisation before evaluation begins. Downstream monitoring cannot recover what was never initialised.
δ not monitored	Drift is occurring but no mechanism is detecting it. The system appears fair — there are no flagged events — but compounds asymmetry beneath the surface.	Implement continuous distributional monitoring. Define explicit θ thresholds in advance. The monitoring function can and should be automated; the

Failure mode	Mechanism	Intervention
	This is the typical failure mode of systems audited only periodically.	threshold is a governance act and cannot be.
κ absent or captured	Drift is detected but correction is blocked, delegated to the wrong function, or proportionality is not enforced. The loop has a sensor but no actuator; the architecture degenerates to surveillance without remedy.	Establish human-governed escalation with clear accountability and timelines. Document the owner of κ . Audit the separation between M and κ ; verify that the same party does not control both.

6.3 Auditability and oversight

The architecture's runtime layer produces, as a by-product of its operation, the artefacts that downstream oversight requires: an immutable record of detected drift events, the threshold against which they were tested, the escalation path each event followed, the human party that authorised any correction, and the timing of correction relative to detection. These artefacts are the substantive content of the human-oversight obligation, and the framework treats their production as a first-order architectural requirement rather than a downstream reporting concern.

7. Cross-Domain Applicability

7.1 Structural universality

The architecture's universality sits in its structure, not its subject matter. The four stages initialise, monitor, threshold, correct and the five conditions are domain-general; what changes between domains is the data, the threshold convention, and the accountable party. The table below specifies the instantiation of the three principal quantities (E_0 , δ , κ) across seven domains.

Domain	E_0 — starting equity	δ — drift to detect	κ — correction
Hiring	Blind, structured scoring	Demographic outcome gaps	Audit and redesign of pipeline stages
Education	Equalised funding and access	Grade and progression gaps	Targeted support and resource allocation
AI model governance	Audited baseline metrics	Model performance drift	Retrain or restrict the model
Health systems	Needs-based access	Treatment disparity	Protocol change

Domain	E_0 — starting equity	δ — drift to detect	κ — correction
Platform moderation	Consistent policy baseline	Enforcement bias signals	Policy recalibration
Criminal justice	Equal procedural rights	Sentencing disparity	Mandatory review
Financial lending	Risk-only assessment criteria	Approval rate by group	Underwriting audit

7.2 The universality claim, qualified

The claim being advanced is structural, not substantive. The framework does not claim that hiring and lending are *the same* they are evidently not, and the political economy of correction differs sharply between them. The claim is that the same operational loop, with domain-specific configuration, is sufficient to monitor and correct drift in either. This is a more modest claim than universality of the underlying ethics, and a more falsifiable one. Section 9 discusses how this claim might be tested empirically.

7.3 The four-question protocol

To instantiate the architecture in a specific system, the operator works through four questions in sequence. The questions are domain-general; the answers are domain-specific.

1	What is E_0 ?	What does fairness look like at the start of this process? What structural advantages must be removed from the inputs before evaluation begins? Who has standing to certify that E_0 has been set?
2	What counts as δ ?	What deviations are being monitored? What signals indicate drift is occurring? What data sources reveal it, and at what cadence? Is the monitoring function structurally separated from the decision function?
3	What is θ ?	At what level of deviation does correction become mandatory? Who has the legitimate authority to set this threshold, and on what evidentiary basis? How often is θ itself reviewed?
4	What is κ , and who governs it?	What correction mechanisms exist? Who is accountable for triggering them? How is correction proportionate to the scale of deviation? Is the corrective party separated from the monitoring party?

8. Regulatory and Standards Alignment

The architecture is constructed to sit beneath, and to operationalise, three families of normative framework now consolidating around high-impact AI: the EU AI Act, the ISO/IEC 42001 management system standard, and the NIST AI Risk Management Framework. The alignments below are structural; they are not claims of compliance, which is the property of an instantiated system in a specific organisational context.

8.1 EU AI Act

- Article 14 (Human oversight). The separation-of-roles principal monitoring may be automated; correction must be human-governed is the architectural realisation of Article 14's requirement that high-risk AI systems be effectively overseen by natural persons during use.
- Article 15 (Accuracy, robustness, cybersecurity). The continuous monitoring stage and the threshold mechanism instantiate the Article 15 requirement that high-risk systems perform consistently throughout their lifecycle and that performance be measurable.
- Article 9 (Risk management system). The three-mode failure taxonomy and the four-question protocol provide a structured residual-risk treatment surface compatible with Article 9's iterative risk management requirement.

8.2 ISO/IEC 42001

- Clause 6 (Planning). The four-question protocol provides a structured input to AI risk and impact assessment requirements.
- Clause 8 (Operation). The runtime control loop is a direct operational instantiation of the AI system lifecycle controls expected under Clause 8.
- Clause 9 (Performance evaluation). The artefacts produced by the runtime layer drift events, threshold records, correction logs feed Clause 9's measurement, monitoring, and internal audit obligations.
- Clause 10 (Improvement). The correction stage κ is the architectural locus of the continuous improvement requirement.

8.3 NIST AI RMF

- GOVERN. The separation of monitoring from correction and the explicit threshold-ownership requirement instantiate GOVERN's accountability obligations.
- MAP. The initialisation stage and the four-question protocol structure the system context, intended use, and risk identification expected under MAP.
- MEASURE. The monitor and threshold stages instantiate continuous measurement of bias and performance drift expected under MEASURE.
- MANAGE. The correction stage and the failure-mode taxonomy provide the structured response surface for MANAGE.

9. Limitations and Future Work

9.1 Limitations of the present contribution

The present paper is a conceptual and architectural contribution; it does not advance an empirical result. Four limitations are stated explicitly.

- The equation $F(t) = E_0 - \sum \delta_i + \sum \kappa_j$ is not mathematically novel. It is an accounting identity rendered in systems-theoretic notation. The framework's contribution does not rest on the equation as a mathematical object; it rests on the operational architecture the equation organises.
- The framework does not select between fairness metrics. The choice between demographic parity, equalised odds, calibration, and individual fairness remains contested in the literature (Chouldechova 2017; Kleinberg, Mullainathan & Raghavan 2017), and the architecture is agnostic to this choice. The architecture specifies the loop, not the metric. This is a deliberate scope decision; a downstream paper may address metric selection within the architecture.
- The architecture has not yet been validated empirically against any deployed system. Appendix A presents a reference implementation against two public datasets, sufficient to demonstrate that the loop executes and produces governance artefacts of the specified form. Empirical validation in a production context is the natural next step.
- The universality claim is structural, not substantive. The claim that the same loop applies across seven domains is a claim about operational compatibility, not about the moral or institutional equivalence of the domains. The literature on what fairness *means* in each domain is large, contested, and not the subject of this paper.

9.2 Future work

Three lines of further work are anticipated.

- Empirical benchmarking. A benchmark study applying the architecture to a deployed system over a multi-month horizon, with comparison against periodic-audit baselines, would test the framework's claim that continuous monitoring with threshold correction reduces consolidated drift relative to retrospective audit.
- Threshold-setting methodology. The framework treats θ as a governance object whose setting is exogenous to the architecture. A complementary contribution addressing the methodology by which θ is set by reference to legal standards, statistical power, or stakeholder deliberation would close a gap the present paper deliberately leaves open.
- Cross-domain transfer studies. A study applying the same reference implementation, with configuration-only changes, across multiple domains would substantiate the structural universality claim made in Section 7.

10. Conclusion

This paper has proposed Equilibrium Fairness as a runtime governance architecture for monitoring and correcting fairness drift in high-impact AI systems. The framework reframes fairness from a static property of a decision rule to a feedback property of a system over time, and specifies an operational loop , initialise, monitor, threshold, correct that connects regulatory principle to deployable practice. Its contribution sits at the implementation layer beneath frameworks such as the EU AI Act, ISO/IEC 42001, and the NIST AI RMF; it does not replace these frameworks and does not claim to resolve the metric debates within technical fairness research. Its claim is more limited and, in consequence, more falsifiable: that a single operational architecture, instantiated by a four-question protocol and audited against a three-mode failure taxonomy, is sufficient to render the human-oversight obligations of contemporary AI governance *operationally tractable* across the high-impact domains where they currently consolidate.

The reference implementation in Appendix A, together with the companion repository, is offered as evidence that the architecture is executable rather than essayistic. The framework's empirical validation, and the testing of its universality claim across additional domains, are work in progress and explicit invitations to collaboration.

Appendix A — Python Reference Implementation

The full source for the reference implementation is provided in a companion repository. This appendix specifies the architecture and reproduces the core modules for review. The implementation targets demonstration, not production: a competent reviewer should be able to read all four modules in twenty minutes and verify that each map to the corresponding stage of the architecture.

A.1 Scope and acceptance criteria

- Two datasets out of the box: UCI Adult (hiring proxy) and UCI German Credit (lending proxy).
- Configurable protected attribute, drift metric, threshold, and window size via YAML config files.
- Two drift metrics: demographic parity gap and equalised odds gap. Pluggable for additional metrics.
- Rolling-window monitoring with configurable window size.
- Structured escalation log (JSON Lines) capturing timestamp, dataset, group, metric, magnitude, breach state.
- pytest suite covering loader, monitor, threshold, and escalation modules.
- Single-command end-to-end run for each dataset.

A.2 Repository structure

```
equilibrium-fairness-reference/  README.md  LICENSE  (MIT)
requirements.txt  configs/  hiring_adult.yaml  lending_german.yaml  src/
__init__.py  loader.py  # E0 – data loading and initialisation
baseline.py  # baseline classifier  monitor.py  #  $\delta$  – drift
detection  threshold.py  #  $\theta$  – escalation check  escalate.py
#  $\kappa$  – human-review trigger and logging  loop.py  # orchestrates the full
loop  notebooks/  01_hiring_demo.ipynb  02_lending_demo.ipynb
03_cross_domain_comparison.md  tests/  test_loader.py  test_monitor.py
test_threshold.py  test_escalate.py
```

A.3 Core module — monitor.py (excerpt)

The monitor stage computes the drift signal δ as a function of recent predictions and protected-attribute labels. The implementation supports demographic parity gap and equalised odds gap as the two reference metrics.

```
"""monitor.py –  $\delta$  computation (stage 2 of the EF architecture)."""
from dataclasses import dataclass
from typing import Literal, import numpy as np
Metric = Literal["demographic_parity_gap", "equalised_odds_gap"]
@dataclass
class DriftReading:
    metric: Metric
    group_a: str
    group_b: str
    magnitude: float
    window_size: int
    n_a: int
    n_b: int

    def demographic_parity_gap(y_pred, group, group_a, group_b):
        """ $P(\hat{y}=1 | A=a) - P(\hat{y}=1 | A=b)$ ."""
        mask_a, mask_b = (group == group_a), (group == group_b)
        if mask_a.sum() == 0 or mask_b.sum() == 0:
            return 0.0, mask_a.sum(), mask_b.sum()
        p_a = float(y_pred[mask_a].mean())
        p_b = float(y_pred[mask_b].mean())
        return p_a - p_b, int(mask_a.sum()), int(mask_b.sum())

    def equalised_odds_gap(y_pred, y_true, group, group_a, group_b):
        """Mean |TPRa - TPRb|, |FPRa - FPRb| across positives and negatives."""
        def rate(pred, truth, m, label):
            m2 = m & (truth == label)
            return float(pred[m2].mean()) if m2.sum() else 0.0
        ma, mb = (group ==
```



```

group_a), (group == group_b)    tpr_gap = rate(y_pred, y_true, ma, 1) - rate(y_pred,
y_true, mb, 1)    fpr_gap = rate(y_pred, y_true, ma, 0) - rate(y_pred, y_true, mb, 0)
return 0.5 * (abs(tpr_gap) + abs(fpr_gap)), int(ma.sum()), int(mb.sum())
def
compute_drift(window, cfg) -> DriftReading:    """Compute  $\delta$  over a rolling window."""
metric = cfg["metric"]    a, b = cfg["group_a"], cfg["group_b"]    if metric ==
"demographic_parity_gap":    mag, n_a, n_b = demographic_parity_gap(
window["y_pred"], window["group"], a, b)    elif metric == "equalised_odds_gap":
mag, n_a, n_b = equalised_odds_gap(    window["y_pred"], window["y_true"],
window["group"], a, b)    else:    raise ValueError(f"Unknown metric {metric}")
return DriftReading(    metric=metric, group_a=a, group_b=b,    magnitude=mag,
window_size=len(window["y_pred"]),    n_a=n_a, n_b=n_b,    )

```

A.4 Core module — threshold.py (excerpt)

```

"""threshold.py —  $\theta$  check (stage 3 of the EF architecture)."""
from dataclasses import
dataclass
from .monitor import DriftReading
@dataclass
class ThresholdResult:    breach:
bool    threshold: float    magnitude: float    direction: str # 'above' or 'within'
def check(reading: DriftReading, theta: float) -> ThresholdResult:    """Return breach
state given drift reading and configured  $\theta$ ."""    mag = abs(reading.magnitude)    return
ThresholdResult(    breach=mag > theta,    threshold=theta,
magnitude=mag,    direction="above" if mag > theta else "within",    )

```

A.5 Core module — escalate.py (excerpt)

The escalate module instantiates κ as a logging-and-flag stub. In a production deployment, the *dispatch* function would notify the human reviewer named in the configuration's *kappa_owner* field. In the reference implementation, dispatch writes a structured event to the escalation log; **no automatic correction is performed**. This is the architectural realisation of monitor $\neq$ decide.

```

"""escalate.py —  $\kappa$  trigger and logging (stage 4 of the EF architecture)."""
import json
from datetime import datetime, timezone
from pathlib import Path
def dispatch(reading,
threshold_result, cfg, log_path: Path):    """Emit an escalation event when  $\theta$  is breached.
Crucially: this function logs and flags. It does not correct.    Correction is the
responsibility of the human party named in    cfg['kappa_owner'].    """    event = {
"timestamp": datetime.now(timezone.utc).isoformat(),    "dataset":
cfg["dataset_name"],    "metric": reading.metric,    "group_a": reading.group_a,
"group_b": reading.group_b,    "magnitude": reading.magnitude,    "threshold":
threshold_result.threshold,    "breach": threshold_result.breach,
"window_size": reading.window_size,    "kappa_owner": cfg["kappa_owner"],
"correction_required": threshold_result.breach,    }    with log_path.open("a") as f:
f.write(json.dumps(event) + "\n")    return event

```

A.6 Orchestration — loop.py (excerpt)

```

"""loop.py — orchestrates the full EF runtime loop."""
import yaml
from pathlib import Path
from .loader import load_dataset, equalise_e0
from .baseline import fit_baseline,
predict_rolling
from .monitor import compute_drift
from .threshold import check
from .escalate import dispatch
def run(config_path: str) -> dict:    cfg =
yaml.safe_load(Path(config_path).read_text())    log_path = Path(cfg["log_path"])
log_path.parent.mkdir(parents=True, exist_ok=True)    # Stage 1: initialise ( $E_0$ )    data
= load_dataset(cfg["dataset"])    data = equalise_e0(data, cfg["e0_policy"])    # Train
baseline on the equalised inputs    model = fit_baseline(data["train"])    # Stages 2-4
over rolling windows    events = []    for window in predict_rolling(model, data["test"],
cfg["window_size"]):    reading = compute_drift(window, cfg)    result =
check(reading, cfg["theta"])    event = dispatch(reading, result, cfg, log_path)
events.append(event)    breaches = [e for e in events if e["breach"]]    return {

```

```
"n_windows": len(events),      "n_breaches": len(breaches),      "log_path":
str(log_path),      }
```

A.7 Configuration example

```
# configs/hiring_adult.yaml dataset: adult      # UCI Adult dataset_name:
hiring_adult e0_policy:  drop_columns: [sex, race] # strip protected attributes from
inputs  protected: sex      # retained as a label for monitoring only group_a:
Male group_b: Female metric: demographic_parity_gap window_size: 500 theta: 0.05
# 5 percentage points kappa_owner: "fairness-review@example.org" log_path:
./logs/hiring_escalation.jsonl
```

A.8 Reproduction

```
# Clone and install git clone <repo-url> equilibrium-fairness-reference cd equilibrium-
fairness-reference pip install -r requirements.txt # Run hiring example python -m src.loop
--config configs/hiring_adult.yaml # Run lending example python -m src.loop --config
configs/lending_german.yaml # Run tests pytest -q
```

Appendix B — Implementation Example: Recruitment

Recruitment is used here as one implementation domain to illustrate the four-question protocol in action. Recruitment is *not* the architecture's defining application; the framework is domain-general, and the present paper situates recruitment as one of seven sectors in which the same loop applies. The choice of recruitment for the worked example is pragmatic: the data is widely available, the harms are well-documented, and the regulatory surface (EU AI Act Annex III; UK Equality Act 2010) is mature.

B.1 Initialisation: E_0 in recruitment

Equalising starting conditions in recruitment means stripping inherited advantage from the inputs to the evaluation stage. Operationally, this requires removal of identifiers (name, address, photograph), removal of features that correlate with identity rather than capability (school prestige, employer prestige), and substitution of structured assessment for unstructured review wherever possible. The protected attribute itself is retained, but only as a monitoring label and never as a model input.

B.2 Drift: δ in recruitment

The most operationally tractable drift signal in recruitment is the demographic parity gap on shortlisting decisions, computed over a rolling window of recent decisions. The equalised odds gap is more informative when ground-truth labels are available (hires, retention, performance), but those labels arrive on a longer cycle than the parity signal and are themselves subject to drift; the architecture monitors both, with separate thresholds.

B.3 Threshold: θ in recruitment

Threshold-setting in recruitment is constrained by legal precedent in the relevant jurisdiction (the four-fifths rule in the United States; selection-rate comparisons under the Equality Act 2010 in the United Kingdom) and by the organisation's internal risk appetite. The architecture requires that θ be set in advance, owned by an identifiable party (typically the head of the relevant talent or compliance function), and reviewed on a defined cycle.

B.4 Correction: κ in recruitment

On threshold breach, κ is typically an audit of the most recent decisions, escalation to a hiring committee with diversity expertise, and a structured intervention in the pipeline (review of screening criteria, recalibration of interviewer training, redesign of the scoring rubric). Crucially, κ is never delegated to the same automated system that produced M ; the separation-of-roles principle is preserved.

B.5 Failure mode diagnosis

Applied diagnostically, the three-mode taxonomy in Section 6.2 produces the following readings for common recruitment failure patterns:

- A pipeline that screens on "culture fit" without structured assessment: **E_0 never set.**
Intervention: redesign the intake.
- A pipeline with structured assessment but only annual diversity reporting: **δ not monitored.**
Intervention: continuous monitoring with explicit θ .
- A pipeline that detects bias signals but routes them to the same hiring manager who produced the decisions: **κ captured.** Intervention: separate the correction function from the decision function.

Appendix C — Glossary

$F(t)$	The system's fairness state at time t . A state variable tracked over operating time, not a scalar property of a decision rule.
E_0	Starting equity. The configuration of inputs at the moment a decision process begins, with inherited advantage removed.
δ	Drift. A measurable distributional deviation from the equitable baseline.
κ	Correction. A governed, accountable, human-authorised action taken in response to a detected drift event.
θ	Threshold. The pre-agreed magnitude of $ \delta $ at which correction becomes mandatory.
M	Monitor. The function (typically automated) that computes δ and tests against θ .
$S(t)$	The state of the system at time t , comprising its current distribution of outcomes.
Negative feedback	A control mechanism in which deviation from a reference range triggers a countervailing response that restores the system toward equilibrium.
Separation of roles	The architectural requirement that M and κ be held by structurally distinct functions; automated monitoring may flag, but only a human-governed function may correct.
Drift event	A discrete observation in which δ has been computed and recorded, whether or not θ was breached.
Escalation event	A drift event in which $ \delta > \theta$; the operational locus of the human-oversight obligation.

References

- Buolamwini, J., & Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of Machine Learning Research*, 81, 77-91.
- Chouldechova, A. (2017). Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big Data*, 5(2), 153-163. <https://doi.org/10.1089/big.2016.0047>
- Dwork, C., Hardt, M., Pitassi, T., Reingold, O., & Zemel, R. (2012). Fairness through awareness. *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, 214-226. <https://doi.org/10.1145/2090236.2090255>
- European Parliament and Council. (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (AI Act). *Official Journal of the European Union*, L, 2024/1689.
- Friedler, S. A., Scheidegger, C., & Venkatasubramanian, S. (2021). The (im)possibility of fairness: Different value systems require different mechanisms for fair decision making. *Communications of the ACM*, 64(4), 136-143. <https://doi.org/10.1145/3433949>
- Hardt, M., Price, E., & Srebro, N. (2016). Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems*, 29.
- International Organization for Standardization. (2023). ISO/IEC 42001:2023 Information technology — Artificial intelligence — Management system. ISO.
- Kleinberg, J., Mullainathan, S., & Raghavan, M. (2017). Inherent trade-offs in the fair determination of risk scores. *Proceedings of the 8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, *Leibniz International Proceedings in Informatics (LIPIcs)*, Vol. 67, 43:1-43:23. <https://doi.org/10.4230/LIPIcs.ITCS.2017.43>
- Kroll, J. A., Huey, J., Barocas, S., Felten, E. W., Reidenberg, J. R., Robinson, D. G., & Yu, H. (2017). Accountable algorithms. *University of Pennsylvania Law Review*, 165(3), 633-705.
- Metcalfe, J., Moss, E., Watkins, E. A., Singh, R., & Elish, M. C. (2021). Algorithmic impact assessments and accountability: The co-construction of impacts. *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*, 735-746. <https://doi.org/10.1145/3442188.3445935>
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT)*, 220-229. <https://doi.org/10.1145/3287560.3287596>
- National Institute of Standards and Technology. (2023). Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1. <https://doi.org/10.6028/NIST.AI.100-1>
- Obermeyer, Z., Powers, B., Vogeli, C., & Mullainathan, S. (2019). Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, 366(6464), 447-453. <https://doi.org/10.1126/science.aax2342>
- Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., & Barnes, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT)*, 33-44. <https://doi.org/10.1145/3351095.3372873>
- Selbst, A. D., Boyd, D., Friedler, S. A., Venkatasubramanian, S., & Vertesi, J. (2019). Fairness and abstraction in sociotechnical systems. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT)*, 59-68. <https://doi.org/10.1145/3287560.3287598>

Veale, M., & Binns, R. (2017). Fairer machine learning in the real world: Mitigating discrimination without collecting sensitive data. *Big Data & Society*, 4(2). <https://doi.org/10.1177/2053951717743530>

Licence

This work is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) Licence.

© 2026 Maphi Bayolo.